Shortest path

Properties of the algorithm

Michel Bierlaire

Optimization: principles and algorithms

ÉCOLE POLYTECHNIQUE
FÉDÉRALE DE LAUSANNE

Properties of the algorithm

The shortest path algorithm calculates labels that verify the optimality conditions. In this video, we review some properties of the algorithm.

Ingredients

Mention that the algorithm stops when $\mathcal{S}$ is empty.

Ingredients

Arcs

c_{ij}

Nodes

λ_i

Set of nodes

$\mathcal{S}$

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \ \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $i \in \mathcal{S}$, then $\lambda_i \neq \infty$.

True at the very beginning:
Initialization: node o
When treating node i, $\lambda_i \neq +\infty$ Any other node inserted after the update of the label.

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

For each node i, the value of λ_i does not increase during the iteration.

Direct consequence of the update condition

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $\lambda_i \neq \infty$, it is the length of one path from o to i.

Initialization: $\lambda_o = 0$ can be interpreted that way.
Iteration 1: labels are the length of the arc = path.
By induction: suppose true before treating node k
because $k \in \mathcal{S}$, $\lambda_k \neq \infty$
Therefore, it is the length of a path (induction assumption)
The iteration adds one arc, if necessary.

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \ \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $i \notin \mathcal{S}$, then

- $\lambda_i = \infty$, or
- $\lambda_j \leq \lambda_i + c_{ij}$, for all j such that $(i,j) \in \mathcal{A}$.

Two reasons not to be in $\mathcal{S}$
1. i has never been treated: $+\infty$
2. i has been treated and removed from $\mathcal{S}$
Just after i has been treated, the inequality is verified.
While i is out of $\mathcal{S}$, the other labels can only decrease, so that the inequality is verified.

Termination

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

Finite number of iterations.

Termination: $\mathcal{S} = \emptyset$

If not, some nodes are added an infinite number of times

Each time, their label has a strictly lower value

$\lambda_i \rightarrow -\infty$

Impossible thanks to the other stopping criterion.

Termination

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If algorithm terminates with $\mathcal{S} = \emptyset$, then $\lambda_j = \infty$ if and only if there is no path from o to j.

Sufficient condition.
Assume by contradiction that there is a path from o to i
The algorithm will set all the labels along the path to a finite number as $\lambda_o = 0$.
Necessary condition: contrapositive of the condition: If $\lambda_i \neq \infty$, it is the length of one path from o to i.

Termination

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \ \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If algorithm terminates with $\mathcal{S} = \emptyset$, then $\lambda_o = 0$.

λ_o cannot increase: $\lambda_o \leq 0$
If $\lambda_o < 0$, there is a cycle with negative cost.
The algorithm will not stop with $\mathcal{S} = \emptyset$

Termination

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If algorithm terminates with $\mathcal{S} = \emptyset$, then if $\lambda_j \neq \infty$, then λ_j is the length of the shortest path from o and j.

p. 565

Termination

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- If $\lambda_j <$ lower bound: STOP,
- otherwise $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If algorithm terminates with $\mathcal{S} = \emptyset$, then for all $j \neq o$ such that $\lambda_j \neq \infty$

$$\lambda_j = \min_{(i,j) \in \mathcal{A}} (\lambda_i + c_{ij}).$$

As the labels are the length of the shortest path, they must verify the optimality conditions:

$$\lambda_j - \lambda_i \leq c_{ij}, \quad \forall (i,j) \in \mathcal{A}.$$

$$\lambda_j - \lambda_i = c_{ij}, \quad \forall (i,j) \in P.$$

Bellman's equation

Draw a picture for one node and explain that we can use them to extract a subnetwork.

Bellman's equation

$$\lambda_j = \min_{(i,j)\in\mathcal{A}} (\lambda_i + c_{ij}).$$

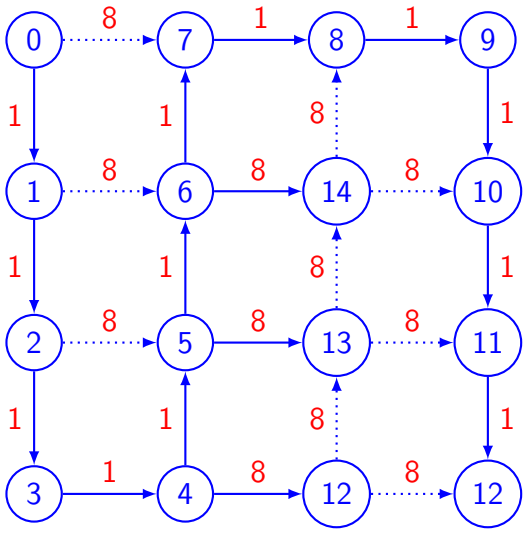
Bellman's subnetwork

- *For each $j \neq o$, select one (i, j) verifying the equation.*
- *If several, choose just one.*
- *Because m nodes, there will be $m - 1$ arcs.*
- *Assume that every cycle in the network (if any) has positive length.*
- *Assume that the Bellman subnetwork contains a cycle. Its length is*

$$c_{i_1 i_2} + c_{i_2 i_3} + \cdots + c_{i_{\ell-1} i_\ell} + c_{i_\ell i_1} = \lambda_{i_2} - \lambda_{i_1} + \lambda_{i_3} - \lambda_{i_2} + \cdots + \lambda_{i_\ell} - \lambda_{i_{\ell-1}} + \lambda_{i_1} - \lambda_\ell = 0,$$

- *So there is no cycle and it is a tree.*
- *Therefore, there is only one path from o to any i.*
- *As the optimality conditions apply, it is the shortest path.*
- *The subnetwork is called the shortest path tree.*

Bellman's subnetwork



Summary

We have seen several important properties of the shortest path algorithm
In particular, the labels are interpreted as the length of path from o.
At the end of the algorithm, it is the length of the shortest path.
Bellman's equations allow us to build the shortest path tree.
It is a spanning tree of the original network.